CSC490 - A2

By: Riyad Valiyev (Student Number: 1009069874), Boquan Zhao (Student Number: 1010419343), Tanayjyot Singh Chawla (Student Number: 1010411109), Carmen Chau (Student Number: 1009043376)

Part 1: Aspirational Datasets

SFX Folder Dataset Schema

Terminology Note: In VocaLocate, a user's SFX library refers to the collection of user-selected root folders being indexed. Each root folder contains SFX files, potentially organized into nested subfolders.

Purpose: Entries here denote user-selected root folders that VocaLocate would search and index to find SFX files best matching user input. VocaLocate recursively scans each root folder for supported SFX files, including those in nested subfolders. Each discovered SFX file is associated with its root folder.

Why it matters: Users may index multiple SFX folders. Representing each folder independently allows VocaLocate to associate SFX files with their respective folders and track indexing progress.

Schema Name: SFX_Folder		
Field	**Type**	**Description**
folder_id	String (UUID)	Unique ID for the folder
folder_path	String	Absolute root path of the folder.
index_status	Enum (Indexing, Pending, Completed, Failed)	Denotes the overall progress of indexing SFX files inside a folder. This entry can provide users with visibility into indexing progress.
last_indexed_at	Timestamp (nullable)	Timestamp of the most recent completed folder index, stored in UTC in the database but converted to the user's timezone when displayed in the UI. Null if no scan has completed. A “nice-to-have” to track better when re-indexing needs to occur.

SFX Sound File Dataset Schema

Purpose: Entries in this schema denote the individual SFX files within user-selected folders. It includes information about the file’s technical properties and optionally semantic metadata if available.

Rationale for including: VocaLocate needs information to identify and locate each SFX file. Technical audio properties guide audio preprocessing to standardize SFX files before converting them into vector embeddings. Ideally, SFX files used for model training and evaluation would also include semantic annotations to support potential model evaluation across different sound categories, although such metadata fields are not expected in real user libraries.

Schema Name: SFX_File		
Field	**Type**	**Description**
folder_id	string (UUID)	Foreign key referencing SFX_Folder.folder_id
sound_id	string (UUID)	Unique ID for the SFX file.
file_path	string	Relative path of the file inside the folder.
audio_format	string	Audio format (eg: WAV, MP3)
sample_rate	int	Entry to help standardize audio format across datasets. Denotes: Number of times per second the continuous audio signal is sampled and converted into a numerical value, measured in Hertz (Hz).
bit_depth	int	Entry to help standardize audio format across datasets. Denotes: Number of bits used to represent the amplitude value of each audio sample.
channel	int	Entry to help standardize audio format across datasets. Denotes: Number of audio stream signals (eg: 1, 2)
duration_sec	float	Length of the SFX file in seconds
category	string (nullable)	Sound category (eg: “Explosion”, “Rattle”)
description	string (nullable)	Free text description about the SFX sound
tags	List of strings (nullable)	Qualitative words describing SFX sound (eg: “Metallic”, “distant”)

Vocal Imitation Dataset Schema

Purpose: Stores metadata about vocal imitation recordings used as search queries for SFX retrieval.

Rationale: Tracking vocalization styles, microphone types, and background noise levels allows us to evaluate how different recording conditions affect retrieval performance and model robustness.

Schema Name: Vocal_Imitation		
Field	**Type**	**Description**
imitation_id	string (UUID)	Unique ID for the imitation recording.
imitation_path	string	Absolute path of the imitation recording.
duration_sec	float	Length of the sound in seconds.
vocalization_type	string	Denotes the type of vocalization style.
device_type	enum	Denotes microphone type. Possible values: Computer microphone, headset microphone, studio microphone, other.
noise_level_db	float	Estimated background noise level of the recording.

Retrieval Query Dataset Schema

Purpose: Entries in this schema represent labelled search queries consisting of a vocal imitation, an optional text hint, and the SFX file that the query is intended to retrieve.

Rationale: An ideal dataset would contain vocal imitation queries paired with their intended SFX files, with some queries also containing short descriptive text hints. These examples would allow us to train and evaluate vocal-imitation retrieval using queries resembling realistic user searches

Schema Name: Retrieval_Query		
Field	**Type**	**Description**
query_id	string (UUID)	Unique ID for retrieval query
imitation_id	string (UUID)	Foreign key referencing Vocal_Imitation.imitation_id
text_descriptor	list of strings (nullable)	Optional list of words provided with the imitation. Eg: [“bell”, “echo”, “ringy”]
target_sound_id	string (UUID)	Foreign key referencing SFX_File.sound_id. Denotes SFX file intended to be retrieved.
split	enum	Dataset partition. Possible values: Train, Validation, Test

QVIM Benchmark Schema

Purpose: Entries in this schema represent baseline QVIM (Query by Voice Imitation) data points used for benchmarking. These are labelled search queries consisting of a vocal imitation and the SFX file that the query is intended to retrieve.

Rationale: This dataset provides a standardized benchmark for evaluating VocaLocate's voice-retrieval model. By evaluating our model on the same QVIM data and using the same metrics as published QVIM baselines, we can directly compare our retrieval performance against established approaches.

Schema Name: QVIM_Benchmark_Data		
Field	**Type**	**Description**
benchmark_query_id	string (UUID)	Unique ID for the QVIM benchmark query.
imitation_id	string (UUID)	Foreign key referencing Vocal_Imitation.imitation_id. Denotes vocal imitations used as the benchmark query
target_sound_id	string (UUID)	Foreign key referencing SFX_File.sound_id. Denotes SFX file intended to be retrieved.
benchmark_split	enum	Predefined QVIM benchmark partition, e.g. Development / Evaluation

Graded Relevant Judgement Dataset

Purpose: Human judgments indicating how relevant candidate SFX are to a given retrieval query. Multiple candidate sounds may be rated for each query, allowing us to evaluate whether the system retrieves useful alternatives even when they are not the exact target.

Rationale: Exact-target evaluation treats every non-target sound as incorrect, even though several SFX may be reasonable substitutes for the same query. Human relevance judgments would let us evaluate whether highly ranked alternatives are still useful and help identify potential false negatives during semi-hard-negative mining.

Schema Name: Relevance_Judgement		
Field	**Type**	**Description**
query_id	string	The query being judged
candidate_sound_id	string	The sound being rated against the query
relevance	int (0 to 3)	0 unrelated, 1 same category, 2 usable substitute, 3 the exact target
rater_id	string	Who gave the rating

Imitator profiles

Purpose: Stores anonymized imitator profiles to support evaluating model performance across users with different backgrounds and vocal imitation styles.

Rationale: This is nice-to-have data for assessing whether one shared model generalizes across different imitators. However, detailed participant information is often unavailable in public datasets and may require additional data collection.

Field	**Type**	**Description**
imitator_id	string (UUID)	Unique, anonymous ID for the person
role	string	Video editor, game audio designer, podcast producer or other
years_experience	int	Years working with sound
first_language	string	First language, since it shapes the sounds people make
music_training	boolean	Whether the person has singing or music training
consent_scope	string	What the person agreed their recordings can be used for

Search session logs

Logs of editors doing the same search tasks with keyword search, reference-audio search and voice search. This is the only data that can answer our first A1 unknown: whether editors find sounds faster with VocaLocate than with the tools they use today.

Field	Type	Description
session_id	string (UUID)	Unique ID for one search attempt
imitator_id	string	The editor doing the search
search_mode	string	keyword, reference_audio, voice or voice_plus_text
query_id	string, nullable	The voice query, if one was used
library_size	int	Number of files in the library being searched
results_shown	ordered list of sound_id	The ranked results the editor saw
previewed_sound_ids	list of sound_id	Results the editor played
chosen_sound_id	string, nullable	The sound the editor picked, empty if they gave up
time_to_choice_sec	float	Time from starting the search to picking a sound
started_at	timestamp	When the search began

Part 2: Reality Check

Category: Vocal Imitation and Reference Sound Datasets

Relevant Item	Description	Commentary
VimSketch [VimSketch DS Link] [VocalSketch Research Paper] [Vocal Imitation Set Research Paper] [VimSketch Research Paper]	A public QVIM (Query by Vocal Imitation) dataset that combines 2 datasets containing paired vocal-imitations and reference sounds: VocalSketch: Crowdsourced vocal imitations of diverse reference audio, with human responses measuring how accurately listeners could identify the imitated sounds.	**Selected.** VimSketch provides several labelled imitation-reference pairs (many imitations mapped to 1 reference sound class per sound category) that can be used to train and fine-tune our MobileNetV3 baseline. Eg: VimSketch only has 1 “dog barking” reference sound, but multiple imitations of dog barks. We can use these labelled pairs as positive examples and combine them with acoustically similar, nonmatching SFX from supplementary datasets to construct training triplets (vocal imitation, positive SFX, negative SFX). This lets us conduct contrastive fine-tuning with

	Vocal Imitation Set: Contains crowdsourced vocal imitations of Freesound recordings, categorized using Google's AudioSet ontology. **VimSketch Total entries:** 542 reference sounds, 12,543 vocal imitations of these sounds (13-37 imitations per reference sound)	semi-hard negatives to improve the baseline's ability to distinguish closely related sounds. Additionally, its sound-category labels and available metadata can help annotate our SFX recordings and construct text descriptors for preliminary testing of CLAP's text-to-audio retrieval capabilities. Limitations: Although VimSketch covers a broad range of sound types, **it contains only one real reference recording per sound class**. More recordings per sound class are ideal to construct finer-grained negative examples for fine-tuning the model to recognize more subtle acoustic distinctions and improve retrieval precision. To address this, we will investigate the non-reference SFX from the Vocal Imitation Set, alongside the ESC-50 dataset.
Vocal Imitation Set [Research paper in above row] [Dataset Link]	(See description in prior row) Entry breakdown: 302 reference recordings with 5,601 quality-assessed vocal imitations. Approximately 2,683 additional non-reference SFX recordings that were not directly imitated, spanning the same sound classes.	**Selected.** We select this dataset on top of VimSketch, despite all (reference) entries overlapping, due to its non-reference SFX. These SFXs help us address the issue with VimSketch where we only had 1 reference SFX per sound class. Thus, they can serve as candidate SFX for semi-hard-negative mining and help our model better distinguish acoustically similar sounds. Furthermore, the dataset's AudioSet-based category labels and available metadata can support SFX annotation and the construction of category-based text queries for testing CLAP's text retrieval component. Limitations: Non-reference recordings lack corresponding vocal imitations, so they cannot independently provide positive training pairs. Additionally, sounds within the same category may still be valid matches, requiring careful negative selection to avoid false negatives.

FSD50K [Dataset Link]	A public Freesound dataset containing 51,197 audio recordings spanning across 200 Google AudioSet ontology sound categories. Includes human-verified sound-event labels identifying which sound categories are present in each recording, alongside descriptions and tags.	**Excluded** FSD50K's human-verified sound-category labels could help identify acoustically related SFX for negative mining. However, unlike our selected datasets (VimSketch, Vocal Imitation Set, and ESC-50-Voice), FSD50K lacks paired vocal imitations and therefore cannot independently provide positive imitation–reference training pairs. Since our selected datasets already provide additional same-class SFX candidates for initial negative mining experiments, we prioritize those sources instead. FSD50K could be reconsidered if we need greater SFX diversity or additional negative candidates. **Limitations:** Potential overlap with VimSketch and VIS due to their shared Freesound origins. Additionally, sounds within the same category may still be valid matches for an imitation, requiring careful negative selection to avoid false negatives.

Category: Evaluation and Testing Datasets

Relevant Item	Description	Commentary
QVIM Challenge Dataset (MR20VI) [Link]	DEV and EVAL data from the AES AIMLA 2025 Query by Vocal Imitation challenge. Each row maps a reference sound to up to three vocal imitation queries; the release is about 450 MB.	**Selected.** This is our held-out benchmark for the vocal-retrieval model and is never used for training or hard-negative mining. Published scores on the development set give us clear numbers to compare against: a mean reciprocal rank (MRR) of 0.044 for random ranking, 0.273 for the MobileNetV3 baseline and 0.293 for the winning challenge entry.
ESC 50 Datasets (broken into 2 components): [SFX Dataset]	SFX Dataset (ESC-50): A public environmental SFX dataset containing 2,000 five-second recordings across 50 sound classes, with 40 recordings per	**Selected.** ESC-50 primarily serves as a small, standardized dataset for smoke-testing our data-processing pipeline. Beneficial to use its fixed five-second SFX recordings, consistent audio format, and structured metadata make

[Vocal Imitations Dataset]	class and predefined evaluation folds. Vocal Imitations (ESC-50-Voice): A supplementary dataset containing 9,920 vocal imitations from 8 Japanese-speaking imitators, paired with 1,240 ESC-50 reference recordings across 31 sound classes (8 imitations per reference recording).	it suitable for reliably testing ingestion, audio cleaning, feature extraction, and caching. Its supplementary ESC-50-Voice dataset provides additional labelled imitation–reference pairs for potential MobileNetV3 fine-tuning and retrieval evaluation. The multiple SFX recordings per sound class could also support any testing or experimentation efforts when evaluating or exploring construction of good negative examples for fine-grained retrieval. Limitations: ESC-50-Voice covers only 31 of ESC-50's 50 sound classes, and its imitations come from eight Japanese-speaking participants, potentially limiting generalization across different imitation styles and backgrounds.
CAPTDURE [Link]	A public audio-caption dataset containing 1,044 single-source environmental sounds and 4,902 corresponding human-written captions (at least three captions per sound). Note: CAPTDURE also provides some multiple-source sound mixtures. However, we will primarily use CAPTDURE's single-source recordings since this type best matches the sound-effect format used in the other datasets (VimSketch, the other Freesound ones, etc)	**Selected.** CAPTDURE provides human-written captions paired with individual environmental sounds, allowing us to evaluate CLAP's text-to-audio retrieval performance. To better reflect VocaLocate's shorter text descriptors, we could use NLP techniques to extract acoustic characteristics (e.g., “metallic,” “sharp,” “distant”) from these captions and manually verify their relevance to the corresponding audio. **Limitations:** The original captions are longer than VocaLocate's intended text hints, and automatically extracted descriptors may lose important context or apply to multiple sounds. We would therefore validate the generated descriptors and supplement them with team-written text hints for evaluating combined vocal-imitation and text retrieval.

freesound-laion-640k [Dataset Link]	Larger Freesound-derived dataset (compared to FSD50K) containing 505,618 audio recordings with uploader-provided titles, descriptions, and tags. Unlike FSD50K, it does not provide standardized, human-verified sound-event labels.	**Excluded**. Our selected datasets (VimSketch, Vocal Imitation Set, and ESC-50) already provide paired imitation–reference recordings and additional SFX candidates for our planned model training, evaluation, and pipeline testing. Therefore, we prioritize these datasets over freesound-laion-640k, whose substantially larger size introduces additional computational, preprocessing, and storage costs. However, should our group want to prioritize having our application support indexing over a more extensive SFX collection, this dataset could provide good files to simulate such settings. Limitations: Audio formats and uploader-provided metadata may vary, requiring additional preprocessing and normalization. Furthermore, the dataset lacks verified imitation–reference pairs and independently validated relevance labels, making it unsuitable as a standalone retrieval-quality benchmark.
Team voice-text-target recordings	Each team member records vocal imitations of a selected set of approximately 50 CAPTDURE single-source SFX recordings, optionally providing short text hints (e.g., “metallic,” “short,” “distant”). Each entry links a vocal imitation, optional text hint, and target SFX.	**Incorporated in pipeline, but data not generated yet.** Provides in-domain voice–text–target queries resembling how users would interact with VocaLocate, supporting evaluation of our hybrid retrieval system. Recordings are reserved for validation and testing, with target SFX assigned to separate splits to prevent data leakage. Validation queries are used to tune the voice/text fusion weight α, while held-out test queries are used for final retrieval evaluation. Current status: Our data-processing pipeline supports ingesting these recordings and assigning validation/test splits, but team

		recordings have not yet been collected or evaluated.

Category: Data we can generate

Relevant Item	Description	Commentary
Mined semi-hard negatives	Hard-negative training examples constructed from the Vocal Imitation Set and/or the ESC 50 dataset. For each vocal imitation, the matched reference sound is used as the positive, while other reference sounds from the same class provide more challenging negative candidates.	**Selected.** Used to further fine-tune the vocal-retrieval model after the initial VimSketch training. The goal is to improve fine-grained discrimination between acoustically similar SFX within the same sound class.
Simulated messy library	A development library built from synthetic held-out SFX, copied into project folders with uninformative filenames. Separate automated tests introduce file additions, edits, moves, and deletions.	**Selected.** Used to test recursive scanning, folder-derived project metadata, incremental processing, duplicate handling, and reuse of cached audio features and embeddings. Folder scans and warehouse rebuilding still occur on each indexing run. It is engineering test data, not a retrieval-quality benchmark.

Part 3: Data-Processing Pipeline:

Our pipeline turns the raw data from Part Two into data our models and app can use through four stages: ingestion, cleaning, transformation, and warehouse loading. These stages support two workflows that were build on the same codebase but serve different purposes.

The dataset pipeline prepares public datasets and team recordings for training, validation, and testing. It is ran on AWS, with shared datasets and processed outputs stored in Amazon S3. Model training is a separate step that consumes these outputs and produces trained model checkpoints. A selected checkpoint must then be distributed with, or downloaded by, the VocaLocate application that end-users would download for use.

The library pipeline runs on the editor's computer as part of the local VocaLocate application. It scans their sound folders, processes new or changed files, and maintains a local data lake and warehouse. The application uses the trained model packaged in its binary to compute library embeddings and match vocal queries to sound effects. The editor's files and index remain local, and using the application does not require access to data or infrastructure on AWS.

Both data-processing workflows are implemented in the pipeline folder of our repository**.**

Pipeline design

Figure 1 shows the whole design. Data moves through three layers of a data lake. Bronze keeps the raw files exactly as we got them, silver holds cleaned audio and tables, and gold holds model-ready features and splits. A DuckDB warehouse sits on top so we can query everything with SQL. The processing code is Python with open-source libraries. On AWS it runs in containers on ECS Fargate, the lake is stored in Amazon S3, Step Functions orchestrates the runs and GPU training runs on Modal (Figure 2). Every AWS resource is defined in Terraform (Part Four). The table below the figures lists each technology.

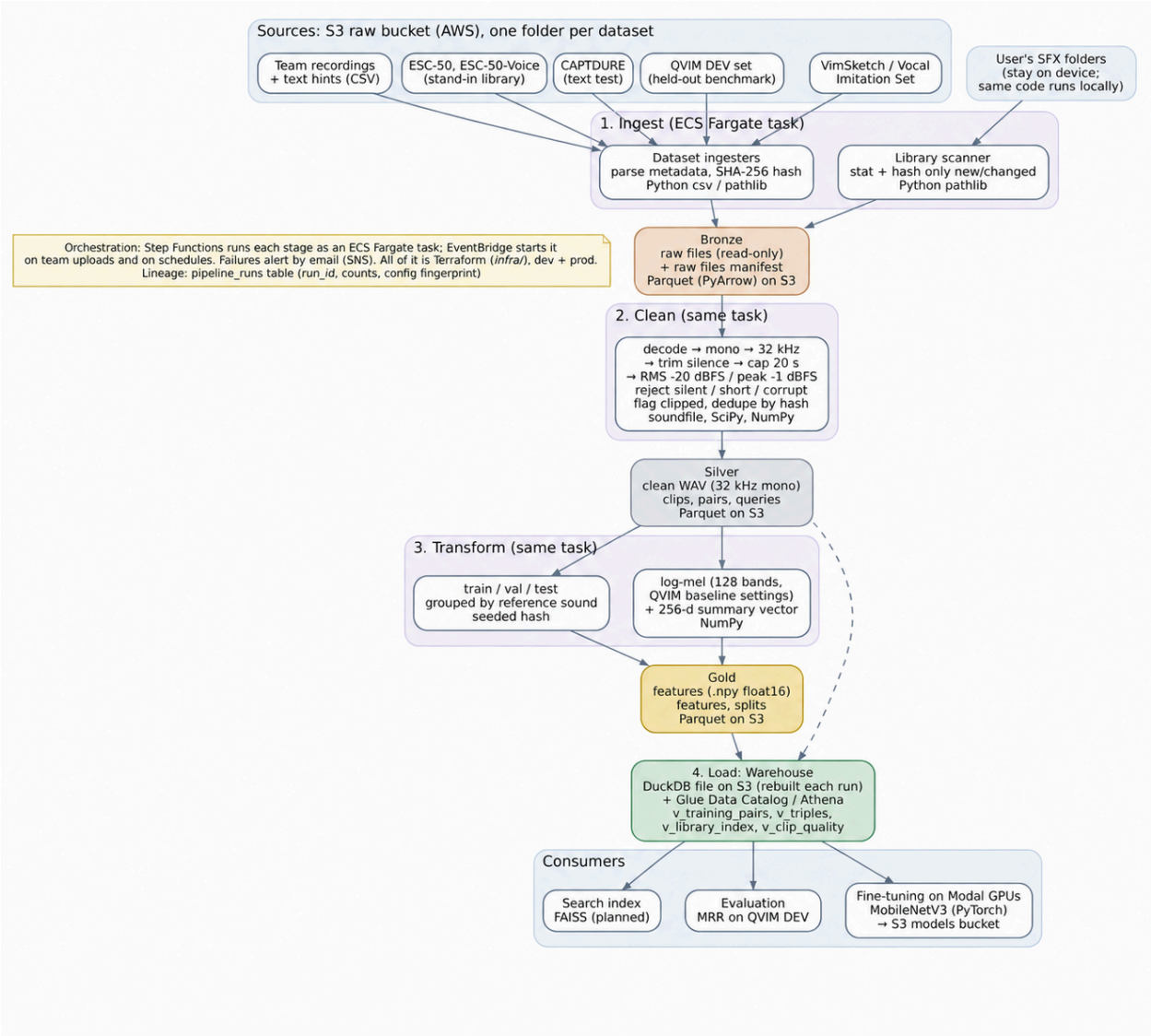


Figure 1: Pipeline architecture, with the technology used at each stage.

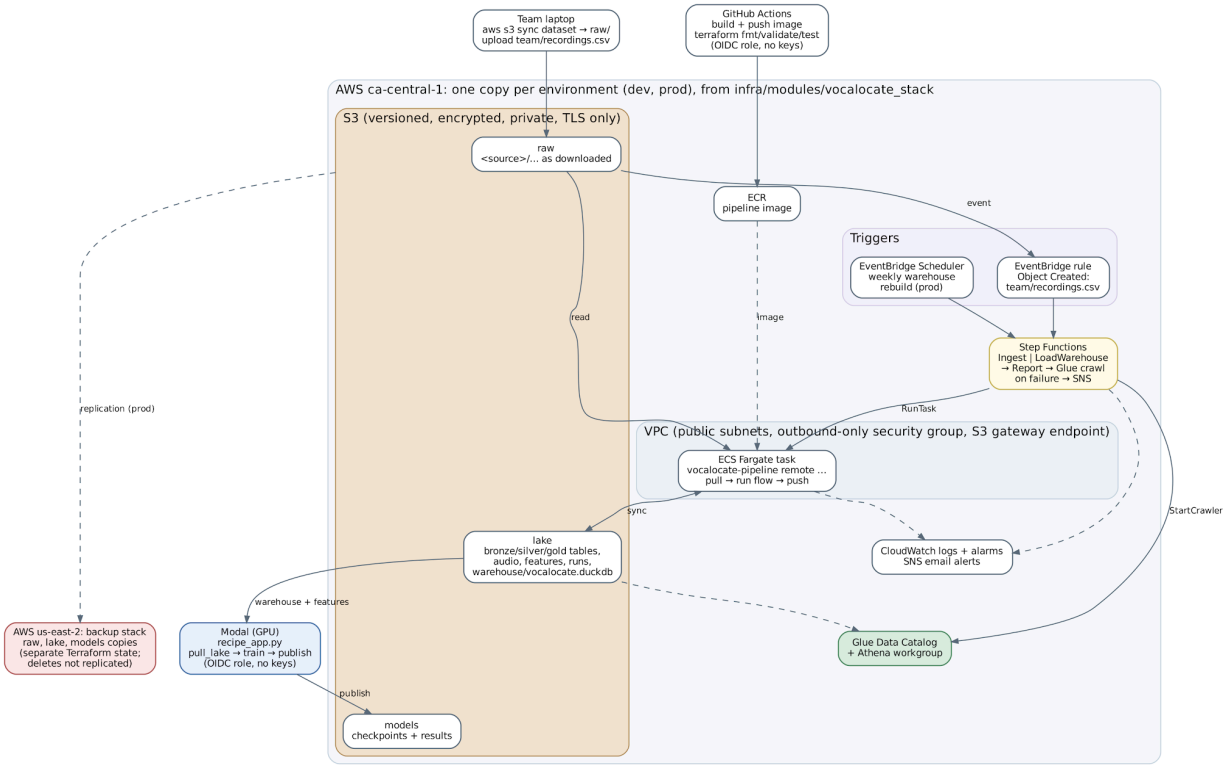


Figure 2: How the pipeline is deployed on AWS (one copy per environment), and where Modal training fits.

The technology used at each stage, and why we chose it:

Stage	Technology	Why we chose it
Ingestion	Python (csv, pathlib, hashlib)	Each dataset ships its own metadata format, so each gets a small reader. A SHA-256 hash of each file gives it a stable ID and shows us duplicates.
Decoding and cleaning	soundfile (libsndfile), SciPy, NumPy	Reads WAV, FLAC, AIFF, OGG and MP3 without FFmpeg. SciPy's polyphase resampler is fast and accurate.
Features	NumPy	We reimplemented the QVIM baseline's log-mel code, so our features match the baseline we reproduce in milestone 1 without putting PyTorch in the pipeline.
Storage	Apache Parquet (PyArrow) on Amazon S3	Columns, compressed and typed, so the schema is enforced when a file is written. One file per table and source. On AWS the lake is an S3 bucket with the same layout; each job copies the parts it needs to its own disk and pushes back what changed.
Warehouse	DuckDB, AWS Glue Data Catalog and Athena	Runs inside our process with no server, reads Parquet directly and works on an editor's laptop as well as on our machines. This fits our local-first

		design. On AWS the DuckDB file is kept in the lake bucket, and Athena queries the same Parquet tables without a server.
Orchestration	AWS Step Functions, EventBridge and ECS Fargate	Each run is a container task on Fargate, so no server runs between jobs. Step Functions retries a task that fails to start and emails the team when a step fails; EventBridge starts runs on new uploads and on a schedule.
Training	Modal (GPU), PyTorch	GPU fine-tuning runs on Modal, one container per training, in parallel. It reads the warehouse and features from S3 and writes models back through an AWS role, so no AWS keys are stored.
Infrastructure	Terraform	Every AWS resource above, and the roles GitHub Actions and Modal use, is defined as code in our repository, with separate dev and prod environments (Part Four).
Testing	pytest, GitHub Actions	Tests build small fake datasets with the same layout as the real ones, so they run in seconds with no downloads.

Ingestion

Each source has an ingester that reads the dataset's own metadata and lists its files. It records each file in a raw_files manifest with its hash, size, label, licence and the reference sound it belongs to. Raw files are never changed. Files from an editor's library are not copied into the lake at all: the manifest points to them where they are, so the library stays on the user's machine, as promised in A1.

- **VimSketch:** an imitation belongs to the reference that has the same file name after the first underscore, the same rule the QVIM baseline uses. Imitations with no matching reference are kept but get no pair.
- **QVIM DEV set:** read from its CSV, which lists up to three imitations per reference. Every clip goes to the test split, so we never train on our benchmark.
- **ESC-50:** read from meta/esc50.csv. Clips cut from the same Freesound recording share a group, so they cannot end up in both training and test data.
- ESC-50-Voice: each of its 9,920 imitations is paired with the exact ESC-50 clip it imitates. Splits are hashed by the clip's Freesound upload, using the same key as the Vocal Imitation Set, so an upload held by both sources lands in the same split.
- Vocal Imitation Set: the category is the AudioSet class and the tags are its ontology path. Rejected and practice takes are kept as unpaired imitations, used only as negatives, and the participant surveys give imitator profiles.
- QVIM evaluation set and CAPTDURE: like the DEV set, every clip goes to the test split. CAPTDURE's captions become text-only queries for the CLAP branch.
- **Team recordings:** a recordings.csv file lists each recording, who made it, the reference sound it imitates and the text hint. Rows with a hint become voice, text and target triples. An optional split column marks each target sound as validation or test, chosen before recording, so team data is never used for training. An optional imitators.csv file next to it holds the imitators' profiles.
- **Editor libraries:** the scanner walks the folders the user chose. The first folder level is used as the project ID, which supports the project tagging from A1.

Cleaning

Imitations recorded on laptops and studio SFX differ a lot in format and level, so every clip goes through the same checks:

Step	Rule	What happens
Decode	File cannot be read	Rejected (decode_error)
Silence	Peak below -60 dBFS	Rejected (silent)
Channels	More than one channel	Averaged to mono
Sample rate	Anything other than 32 kHz	Resampled to 32 kHz, the rate used by the QVIM baseline and its pretrained weights
Trim	Start or end more than 50 dB below the peak	Trimmed
Too short	Under 0.1 s after trimming	Rejected (too_short)
Too long	Over 20 s	First 20 s kept and flagged as truncated. thatsoundslikeme drops these clips; we keep them so long ambiences in an editor's library are still searchable.
Loudness	Any level	Scaled to -20 dBFS RMS, with the peak capped at -1 dBFS, so laptop imitations and studio SFX end up at a similar level
Clipping	1% or more of samples at full scale	Kept and flagged, since a clipped SFX can still be the sound the editor wants
Duplicates	Same SHA-256 as an earlier file in the same source	Marked as a duplicate and not processed again

Rejected and flagged clips stay in the clips table with the reason, so we can count problems per source (the v_clip_quality view) instead of losing them silently.

Transformation

- **Log-mel spectrograms:** 128 mel bands, a 1,024-point FFT, an 800-sample window, a 320-sample hop, pre-emphasis and a 0 to 15 kHz range. These are the QVIM baseline's settings, so our features match theirs. They are saved as float16 to halve the storage.
- **Summary vectors:** the mean and standard deviation of each mel band, 256 numbers per clip. This is the simple non-neural baseline from milestone 1, and it lets us test search before the neural models are ready.
- **Splits:** each reference sound goes to train (80%), validation (10%) or test (10%) based on a seeded hash of its ID, and all of its imitations follow it. The hash does not change when new files arrive, so a sound never moves between splits. This is the split rule from milestone 2. Team recordings are never used for training: each team target sound goes to validation or test, as set in recordings.csv or by a seeded 50/50 hash, so the fusion weight α is tuned on validation queries and tested on the rest.
- **Pairs and triples:** imitations are linked to their reference sound (pairs), and team text hints are linked to their imitation and target sound (queries).

The training code still reads the clean audio, because the baseline computes its features on the GPU with random masking during training. The stored features are for evaluation, the non-neural baseline and quick checks.

Library pipeline

Editors keep adding sounds, so the library pipeline only does the work that is needed (Figure 3). It compares each file's size and modification time with what it saw on the last run, and only hashes files that are new or changed. If a hash is already known, the file was moved or renamed, and its clean audio and features are reused. Files that have disappeared are marked as deleted, not removed, so their history stays. On our test library, a second run over unchanged files does no audio processing at all.

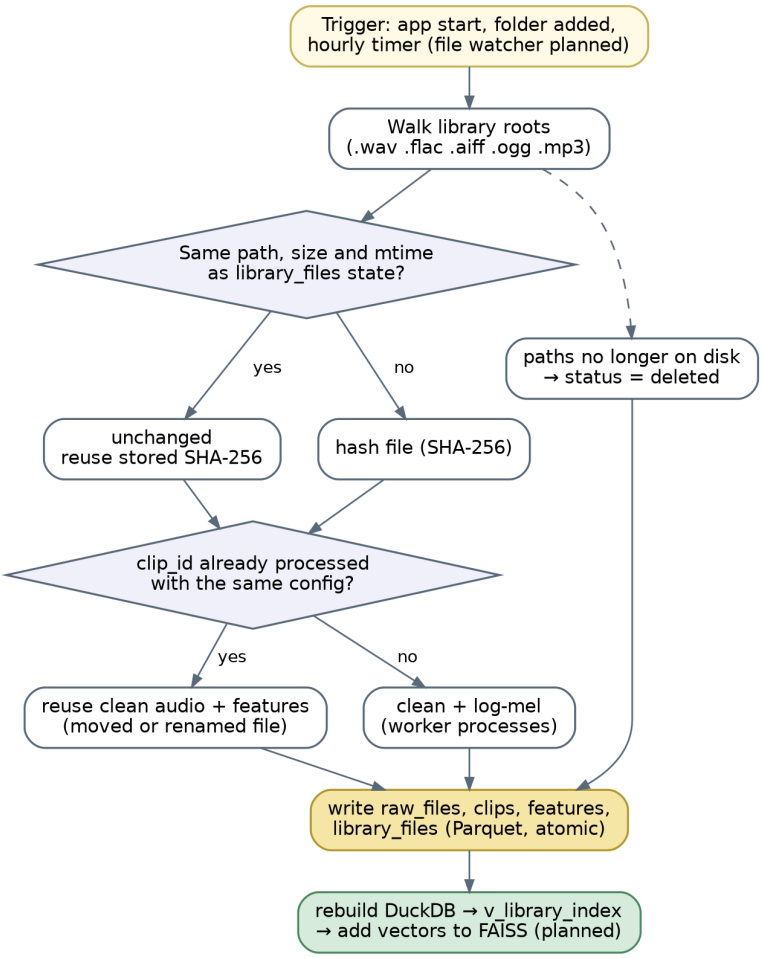


Figure 3: How the library pipeline decides what to process.

Data lake and warehouse design

The lake is a folder tree with the same layout on a laptop and on AWS, where it is an S3 bucket in each environment. On AWS the raw downloads are kept in a separate raw bucket, one folder per source, and trained models in a models bucket:

```
lake/
  bronze/<source>/...                    raw downloads, never changed
  bronze/tables/raw_files/<source>.parquet
  silver/audio/<source>/<id>.wav         clean 32 kHz mono audio
  silver/tables/{clips,pairs,queries,sfx_metadata,captions,imitators}/<source>.parquet
  gold/features/<source>/<id>.npy        log-mel features
  gold/tables/{features,splits,embeddings,hard_negatives}/<source>.parquet
  state/library_files/<library>.parquet  what the scanner sees
  state/library_roots/<library>.parquet  the folders each library indexes
  runs/<run_id>.parquet                  one row per pipeline run
  warehouse/vocalocate.duckdb
```

Each table is stored as one Parquet file per source. Re-running one source replaces only its own files, and every file is written under a temporary name and then renamed, so a crash never leaves only half the file. After each run the DuckDB warehouse is rebuilt from the lake. The warehouse file is stored in the lake bucket, and a Glue crawler registers the same Parquet tables in the Glue Data Catalog, so they can also be queried with Athena without downloading anything.

The warehouse adds these views:

View	What it gives us	Used by
v_training_pairs	Clean imitation and reference pairs with their split	Fine-tuning (milestones 2 and 3)
v_triples	Imitation, text hint and target sound, across sources	Tuning the fusion weight α
v_library_index	Active library files with their project, clean audio and features	The search app and the FAISS index (milestone 4)
v_clip_quality	Counts of clean, rejected and flagged clips per source	Data-quality checks after each run
v_cross_source_duplicates	The same audio in more than one source. A second view, v_shared_recordings, finds one original recording (e.g. a Freesound upload) held by more than one source, with the splits it landed in, so no recording ends up in both train and test	Removing overlap, e.g. between VimSketch and the Vocal Imitation Set
v_sfx_candidates	Clean reference and library clips of the datasets, with their split, features and metadata	Candidate SFX for negative mining and evaluation
v_unpaired_imitations	Imitations kept only as negatives, with the reference they imitate	Training negatives that never form a pair
v_captions	CAPTDURE captions with their target clip and split	Testing the CLAP text branch
v_hard_negatives	Mined negatives with the category of the target and of the negative	Comparing the mining strategies (milestone 3)
v_sfx_folder, v_sfx_file, v_vocal_imitation,	One view per Part One schema, with exactly its column names and NULL	Checking our data against Part One (see the Part 1 to Part 3

v_retrieval_query, v_qvim_benchmark, v_imitator_profile	where we have no data	Mapping tab)

Data schemas

These are the tables the pipeline writes, defined in schemas.py. The IDs match the wish-list schemas from Part One: clip_id plays the role of sound_id and imitation_id, and imitator_id and query_id are the same. Because clip_id is the SHA-256 hash of the raw file, the same file always gets the same ID. Types are enforced when each Parquet file is written. The pipeline also checks that required fields have no missing values and that fields such as role, status and split only hold allowed values. Each Part One schema that our data can fill also has a warehouse view with exactly its column names (in the views table above), and the Part 1 to Part 3 Mapping tab compares every Part One field with these tables. Part One lists UUIDs; we use the SHA-256 instead so that IDs never change. The fields no dataset can fill are listed at the end of this section.

raw_files (bronze): one row per raw file

Field	Type	Description
source, source_version	string	Dataset name and version, e.g. vimsketch, 1.0
source_key	string	Path of the file inside the dataset
uri	string	Where the raw bytes are stored
file_sha256, size_bytes	string, int	Content hash (becomes clip_id) and file size
role	string	reference, imitation, unpaired_imitation or library
label	string, nullable	Class or category given by the dataset
group_key	string, nullable	The reference sound this file is, or imitates
imitator_id, project_id	string, nullable	Who recorded an imitation; which project a library file belongs to
licence	string, nullable	Licence of the file or dataset
extra_json	string, nullable	Source-specific metadata, e.g. the ESC-50 fold
ingested_at, run_id	timestamp, string	When and by which run the file was recorded

clips (silver): one row per raw file after cleaning

Field	Type	Description
clip_id	string	SHA-256 of the raw file
source, source_key, role, label, group_key, imitator_id, project_id, licence	as in raw_files	Copied from the manifest
status	string	ok, rejected or duplicate

reject_reason	string, nullable	e.g. silent, too_short, decode_error, duplicate_of:<file>
orig_sample_rate, orig_channels, orig_duration_sec	int, int, float	Audio format before cleaning
duration_sec, peak_dbfs, rms_dbfs	float	Measured after cleaning
truncated, clipped	bool	Quality flags
clean_path	string, nullable	Path of the clean 32 kHz mono WAV
config_fingerprint, run_id	string	Hash of the cleaning and feature settings, and the run that wrote the row
orig_format, orig_bit_depth	string, int, nullable	Format of the original file (WAV, FLAC, MP3, ...) and its bit depth, empty for lossy formats
noise_floor_dbfs	float, nullable	Background noise of the original recording: the 10th percentile of 50 ms frame loudness

pairs and queries (silver)

Field	Type	Description
pairs: imitation_clip_id, reference_clip_id	string	An imitation and the reference sound it imitates
pairs: source, group_key	string	Dataset and reference sound
queries: query_id, source	string	ID of a voice, text and target triple (a hash of the file and its hint), and its dataset
queries: imitation_clip_id	string	The imitation
queries: text_hint, hint_type	string, string (nullable)	The hint and what it describes, e.g. "metallic", material. The hint is kept as typed; v_retrieval_query splits it on commas into a list of terms
queries: target_group_key	string	The target sound, which can be in another source
queries: target_clip_id	string, nullable	The exact target sound, when its group has a single reference
queries: split	string, nullable	val or test for team recordings, chosen before recording; other queries follow their target's split

features and splits (gold)

Field	Type	Description
features: clip_id, source, feature_version	string	The clip, its dataset and the feature recipe (logmel-qvim-v1)

features: feature_path, n_frames	string, int	The saved log-mel array and its length
features: summary	list of 256 floats	Mean and standard deviation of each mel band
splits: clip_id, source, source_key, group_key	string	The clip, where it comes from, and the reference sound that decides its split
splits: split, strategy	string	train, val or test, and how it was assigned: group hash, benchmark (always test), or team (the split from recordings.csv, or a val/test hash)

sfx_metadata (silver): optional semantic metadata per SFX

Field	Type	Description
clip_id, source	string	The clip and its dataset
category	string, nullable	Sound class, the most specific reliable label the dataset gives, e.g. Dog
description	string, nullable	Free-text description of the sound
tags	list of strings, nullable	Descriptive words, e.g. metallic, distant
recording_id	string, nullable	Shared by clips cut from the same original recording, e.g. a Freesound ID

captions (silver): text-only queries for the text branch

Field	Type	Description
caption_id, source	string	ID of the caption and its dataset (CAPTDURE)
clip_id	string	The clip the caption describes, which is also its target
caption, language	string	The caption and its language
annotator_id	string, nullable	Who wrote the caption

embeddings and hard_negatives (gold)

Field	Type	Description
embeddings: clip_id, source	string	The clip
embeddings: model_version, tower	string	The model, and which encoder: sfx or imitation (the two QVIM towers) or clap_audio
embeddings: vector	list of floats	The embedding, cached so

		re-indexing only embeds new clips
hard_negatives: strategy, model_version	string	How the negatives were mined (retrieval, metadata or random) and the model that scored them
hard_negatives: imitation_clip_id, positive_clip_id, negative_clip_id	string	The imitation, its target and the mined wrong sound
hard_negatives: negative_rank, positive_score, negative_score	int, float, float	Where the model ranked the negative, and both scores
hard_negatives: selection, relation, run_id	string	band, fallback or random; how metadata relates the negative to the target; the mining run

library_files (state): what the library scanner has seen

Field	Type	Description
library, path	string	Library name and absolute file path
size_bytes, mtime_ns	int	Used to skip unchanged files without hashing them
file_sha256	string	Content hash; matches clip_id
project_id	string, nullable	First folder under the library root
status	string	active or deleted
first_seen, last_seen	timestamp	When the scanner first and last saw the file
root_id, rel_path	string, nullable	The indexed folder the file was found under (library_roots) and its path inside that folder

library_roots (state): one row per folder a library indexes

Field	Type	Description
root_id	string	First 32 hex digits of the SHA-256 of the folder path (Part One's folder_id)
library, root_path	string	Library name and the folder's absolute path
index_status	string	pending, indexing, completed or failed. The pipeline writes completed or failed; the app will write the other two.
last_indexed_at, last_run_id	timestamp (nullable), string	When the folder was last fully indexed, and the latest run

imitators (silver)

Field	Type	Description
imitators: imitator_id, source	string	Matches clips.imitator_id. From the Vocal Imitation Set and VocalSketch surveys and the team's imitators.csv
imitators: role, years_experience	string, int, nullable	e.g. video editor or crowd worker; years working with sound
imitators: first_language, music_training, consent_scope	string, bool, string, nullable	As in Part One's imitator profiles

pipeline_runs: one row per run, for lineage and monitoring

Field	Type	Description
run_id, flow, source	string	e.g. dataset-20261002T143620Z-00b9f3, dataset, esc50
started_at, finished_at	timestamp	When the run started and ended
status, message	string	succeeded or failed, with the error message if it failed
n_input, n_processed, n_reused	int	Files seen, newly processed and reused from earlier runs
n_ok, n_rejected, n_duplicate	int	Cleaning outcome
config_fingerprint	string	The settings used

Part One fields we struggled to populate

To connect our concrete data schemas in Part Three with the aspirational schemas outlined in Part One, we created warehouse views that preserve most of the original schema property names and intended behaviour. The exceptions are Relevance Judgement and Search session logs, which are not implemented in Part Three, and Imitator profiles, whose view has many missing properties.

Sometimes certain fields remain unavailable, approximated, or unimplemented because the datasets and generated data discussed in Part Two cannot feasibly provide all the required information (or because contextually there is little fit). The table below summarizes these gaps and explains why they are not fully addressed in our current implementation.

Part One schemas or fields	How we handle in Part 3	Why the gap exists
SFX_Folder	Folder information is tracked for users' local SFX libraries, but	Public datasets are not organized as user-selected library folders.

	not for public datasets.	Folder records are instead generated when users index their own libraries.
SFX_File.bit_depth for MP3, Vorbis and Opus files	Stored where available; otherwise left empty.	Certain compressed audio formats (e.g., MP3) do not have a directly applicable PCM bit depth.
SFX_File.category, description and tags	Stores available sound categories and descriptive metadata, but coverage varies across datasets (eg: For some datasets, their entries have categories but little to no tags) Moreover these fields are not present in user library fields (which is an intended design choice as we expect VocaLocate to give recommended SFX matches based on user vocal imitation and entered text descriptions only.	Not all datasets provide detailed semantic annotations, and users' SFX libraries may have uninformative filenames or missing metadata.
Vocal_Imitation.vocalization_type and device_type	Not collected or stored	Our selected datasets generally do not record vocalization styles or microphone types. These are nice-to-have annotations for evaluating model robustness, rather than essential retrieval information.
Vocal_Imitation.noise_level_db	Estimates background noise from the audio recording.	Datasets generally do not include calibrated background-noise measurements, so we approximate them using audio analysis
Retrieval_Query.text_descriptor	Uses available short text hints, sound categories, or tags from dataset metadata CSV files or SFX file name to construct queries. Some dataset do not come with a separate metadata file.	Most vocal-imitation datasets lack paired text queries, so additional text hints must be generated or collected. The current processing also has limited support for extracting descriptors from full sentences.
QVIM_Benchmark_Data.benchmark_query_id	Uses the vocal imitation's clip_id, a unique identifier	The QVIM dataset does not provide separate query IDs.

	generated from the audio file's contents using SHA-256, as the benchmark query ID.	Since each benchmark query corresponds to a vocal imitation recording, its clip_id can uniquely identify the query without generating an additional ID.
Relevance_Judgement, all fields	Not currently implemented, although some datasets provide related human evaluation scores	The Vocal Imitation Set provides 0–100 scores evaluating how closely vocal imitations resemble their intended SFX, while VocalSketch measures listeners' ability to recognize imitated sounds. However, neither provides relevance ratings for multiple candidate SFX retrieved for a given query (e.g., exact matches versus usable alternatives). Collecting these judgments would require additional human evaluation, which is outside our current scope.
Imitator profiles.first_language and consent_scope	Stores available participant information, but detailed profiles are incomplete across datasets. ESC-50-Voice gives only first language (all eight imitators are Japanese speakers); the Vocal Imitation Set and VocalSketch surveys give role and years of music and audio experience.	Many datasets omit characteristics such as first language and recording consent details. Additional information could be collected from team participants.
Search session logs	Not stored	These records require actual user interactions with VocaLocate. They could be collected through the application should we conduct usability studies in future milestones.

When the pipelines run

Most pipelines run when their input changes rather than on a timer. The library pipeline is the exception, because it has to keep up with folders the user keeps changing. Re-runs are cheap because finished work is cached by file hash and settings. On AWS each dataset run is a container task started by Step Functions, and EventBridge starts runs by itself when new team recordings are uploaded or on a schedule. The library pipeline's triggers come with the app (see Next steps).

Pipeline	When it runs	Use case
Dataset: VimSketch	Once after the download is copied to the S3 raw bucket (started by hand in Step Functions), then again when the dataset version or our cleaning or feature settings change (the config fingerprint changes)	Training pairs for the two MobileNetV3 encoders (milestones 1 to 3)
Dataset: QVIM DEV set	Same as VimSketch	Held-out benchmark. We report MRR against the published scores.
Dataset: ESC-50	By hand in the dev environment before merging pipeline changes (about 5 seconds for 100 clips)	Test on real audio and a small stand-in library
Dataset: team recordings	Automatically when recordings.csv is uploaded to the raw bucket after each recording session (an EventBridge rule), weekly while we record	Voice, text and target triples for tuning α, and a test set recorded the way users will record
Library	On the editor's computer: when the app starts, when a folder is added and every hour; later on every file change	Keeps the search index up to date without reprocessing the library (milestone 4)
Warehouse rebuild	At the end of every run, and every Monday in prod (EventBridge Scheduler)	Training, evaluation and the app always read the newest data
Tests (CI)	Every push that changes the pipeline or the infrastructure (the Terraform checks)	Catch broken cleaning rules or schema changes before they reach our data
Training on Modal (GPU)	By hand after a dataset run. It reads the warehouse and features from the S3 lake and writes checkpoints and results to the models bucket	Fine-tuning experiments (milestones 2 and 3)

Initial implementation

The first version is in the pipeline folder of our repository. It is a Python package with a command-line tool:

```
pip install -e pipeline
vocalocate-pipeline ingest-dataset --source vimsketch --path Vim_Sketch_Dataset
vocalocate-pipeline index-library --root ~/SFX --name my_library
vocalocate-pipeline report

vocalocate-pipeline remote ingest --source esc50   # on AWS: the same flow against the S3 lake
```

What works today:

- Ingesters for VimSketch (with VocalSketch's extra imitations), the Vocal Imitation Set, the QVIM DEV and evaluation sets, ESC-50, ESC-50-Voice, CAPTDURE, team recordings and any dataset that ships a metadata CSV, plus the incremental library scanner.
- All the cleaning rules above, log-mel features, summary vectors, grouped splits, pairs and triples.
- The Parquet lake, the DuckDB warehouse with its 16 views, including one per Part One schema

we can fill, and a pipeline_runs row for every run.
- The AWS deployment: the same flows run as ECS Fargate tasks against an S3 lake, started by Step Functions, with dev and prod environments defined in Terraform (Part Four).
- 83 tests on fake datasets that copy the real folder layouts, including the S3 sync against a local S3 server, run by GitHub Actions on every push that changes the pipeline.

We also ran it on 100 real ESC-50 clips. With four worker processes the first run took about 5 seconds; all 100 clips passed cleaning and 2 were flagged as clipped. A second run reused every clip and took about 1.5 seconds. Indexing the same folder as a library gave 100 entries in v_library_index, and v_cross_source_duplicates found that the library and the ESC-50 source hold the same files.

Next steps

These features are designed for but not built yet:

Next step	**Why**
Expand support for more robust text queries in evaluation datasets	Our current dataset pipeline primarily constructs text queries as a collection/list of short descriptors, sound categories, and tags rather than actual sentences. Furthermore, our evaluation dataset for text queries (CAPTDURE) provides longer audio text queries. To better reflect realistic user inputs, we plan to generate or collect more varied natural-language text queries paired with their target SFX, including queries combined with vocal imitations. This would allow us to more thoroughly evaluate CLAP's text retrieval performance and tune and test our hybrid voice–text retrieval system on representative user queries.
Collect team-generated vocal imitation to pair with existing SFX reference files	Many available SFX recordings lack corresponding vocal imitations, limiting their use as positive training pairs. We plan to record our own vocal imitations for selected unpaired SFX from existing datasets, prioritizing underrepresented sound categories. This could also allow us to incorporate additional SFX-only datasets, such as FSD50K, expanding the diversity of paired examples available for model training and evaluation.
Incremental FAISS indexing for evolving SFX libraries	Extend our existing library pipeline's file-hashing and caching mechanisms to reuse embeddings for unchanged SFX files and generate new embeddings only when necessary, avoiding redundant processing. Integrate these embeddings into a local FAISS index, investigating incremental updates and whether periodic rebuilding is required depending on the indexing strategy. Additionally, implement a file watcher to detect

	library changes and reduce reliance on periodic folder scanning.
Hard-negative mining job	Explore strategies for identifying suitable hard-negative SFX from our selected datasets, including acoustically similar recordings within the same sound category. We plan to compare retrieval-based and metadata-based negative selection to determine which examples are most useful for fine-tuning MobileNetV3, while minimizing false negatives (i.e., sounds that may actually be valid matches). Our demo already supports preliminary hard-negative mining, but further experimentation and integration into the dataset pipeline are needed.

Part Four: Infrastructure as Code

We deploy the Part Three pipeline to AWS with Terraform. All of the infrastructure code is in the infra folder of our repository. It builds two environments, dev and prod, from the same modules, plus a separate backup stack for the disaster-recovery demo in Part Five. The library pipeline and the search app stay on the editor's computer, as we proposed in A1, so user SFX files and their search index therefore remain on-device and are not uploaded to AWS, and AWS holds only project datasets, the processed lake and the trained models. GPU training runs on Modal, which reads from and writes to the same S3 buckets.

What we deploy

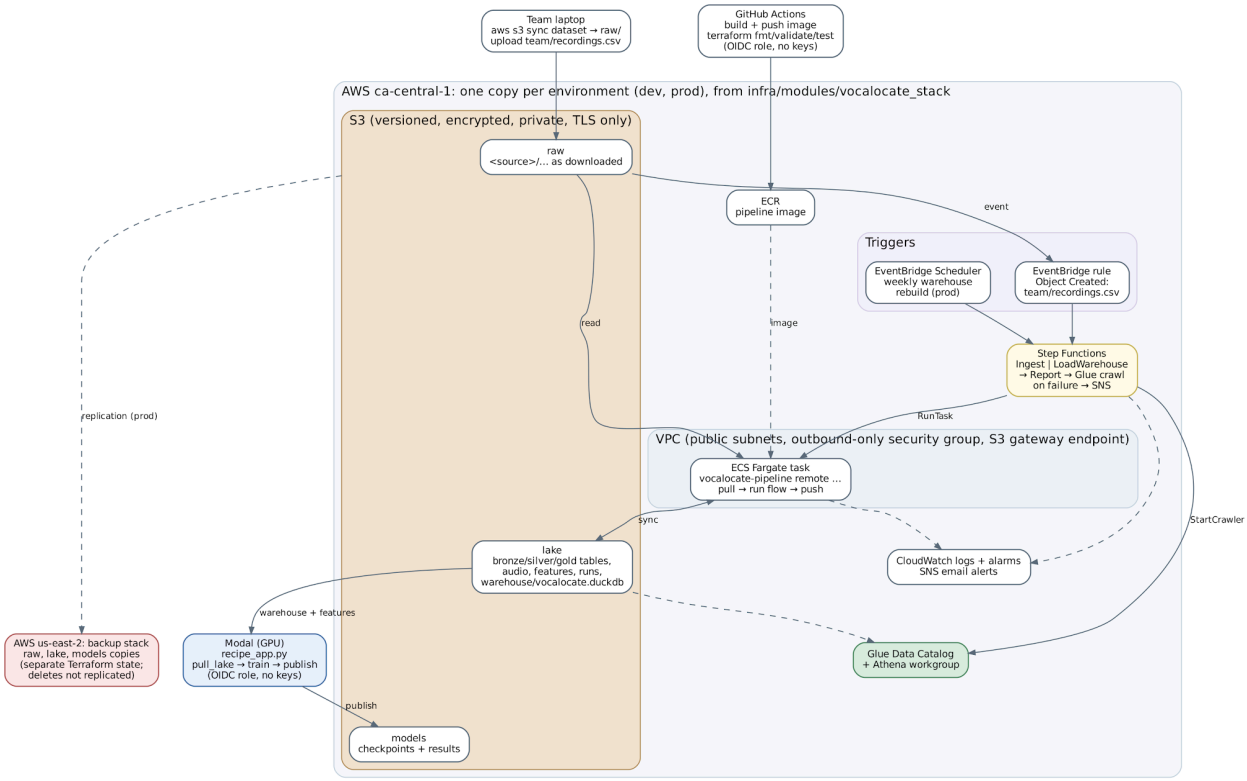


Figure 4: How the pipeline is deployed on AWS (one copy per environment), and where Modal training fits.

Figure 4 shows how the pieces connect. Each part of the Part Three design runs on one AWS service, defined in one of our Terraform modules:

Part Three piece	AWS resource	Module
Raw datasets and team recordings	S3 raw bucket, one folder per source	data_lake
Data lake: bronze, silver and gold tables, clean audio, features, pipeline_runs	S3 lake bucket, with the same layout as the local lake folder	data_lake

Trained models and results	S3 models bucket	data_lake
Ingestion, cleaning, transformation and loading	ECS Fargate task running the pipeline container image, stored in ECR	pipeline_compute
Network for the tasks	VPC with two public subnets, an outbound-only security group and an S3 gateway endpoint	network
When the pipelines run	Step Functions state machine, an EventBridge rule on team uploads and EventBridge Scheduler	orchestration
Warehouse	DuckDB file in the lake bucket, plus a Glue Data Catalog and an Athena workgroup	warehouse_catalog
Data-quality checks and alerts	A report step after every run, CloudWatch alarms and SNS email	orchestration, monitoring
Access for CI and GPU training	IAM roles for GitHub Actions and Modal, assumed with OIDC tokens	access
Terraform state, budget and identity providers	S3 state bucket, AWS Budgets, IAM OIDC providers	bootstrap (once per account)
Backups of prod data	Versioned S3 buckets in us-east-2	backup (separate state)

How the code is organized

The infra folder has four Terraform roots and the modules they share:

```
infra/
  bootstrap/          state bucket, OIDC providers, budget (once per account)
  backup/             prod backup buckets in us-east-2, with their own state
  envs/dev/           dev: backend, provider and settings
  envs/prod/          prod: backend, provider and settings
  modules/
    vocalocate_stack/   everything one environment needs
    bucket/           a private, encrypted, TLS-only S3 bucket
    network/, data_lake/, pipeline_compute/, orchestration/,
    warehouse_catalog/, monitoring/, access/
```

- **Prod and Dev environments:** envs/dev and envs/prod each hold only a backend, a provider and one call to the vocalocate_stack module with their own settings. Every resource is defined once, so the two environments cannot drift apart.
- **Our own modules:** every component is a module we wrote. The bucket module is used for every S3 bucket in the project (the lake buckets, the backups, the state bucket and the Athena results bucket), so all of them get the same encryption, public-access block and TLS-only policy.
- **No provisioning scripts:** everything that builds infrastructure is Terraform. The two steps outside it only move data or settings: copying a dataset into the raw bucket with aws s3 sync, and creating the Modal secret with the Modal CLI, for which Terraform prints the exact command as an

output.

- **Shared state:** each root keeps its state in the bootstrap bucket, which is versioned and uses S3 lock files, so two teammates cannot apply at the same time and an earlier state can be recovered.
- **Names and tags:** resources are named vocalocate-<environment>-<part> and tagged with the project, the environment and ManagedBy=terraform. Bucket names end in the account ID so they are globally unique.

Environments

Dev is a small, disposable copy for trying pipeline and infrastructure changes on ESC-50. Prod holds the data we train and report on, so it is larger, refuses to delete data and is backed up.

Setting	dev	prod
Fargate task	1 vCPU, 4 GB memory, 30 GB disk	4 vCPU, 16 GB memory, 100 GB disk
Pipeline config	configs/dev.toml, 1 worker	configs/prod.toml, 4 workers
Destroying buckets that hold data	Allowed	Refused, unless force_destroy is set for the disaster-recovery demo
Earlier object versions kept	7 days	30 days
Backup to us-east-2	None	All three buckets
Schedules	None	Warehouse rebuild and report every Monday at 6:00, Toronto time
Logs kept	7 days	90 days
Modal environment allowed to train	dev	main
VPC address range	10.40.0.0/16	10.41.0.0/16

Both pipeline configs keep the audio, feature and split settings from Part Three, and a test checks that they do, so features made in dev and in prod are the same.

How a pipeline run works on AWS

The Part Three flows work on files on disk: they write each file under a temporary name and then rename it, and DuckDB reads a local file. Rather than rewrite them for S3, each Fargate task works on a local copy of the lake:

- **Pull:** the dataset from the raw bucket, plus the lake's tables and that source's clean audio and features, so the content-hash cache from Part Three still skips clips an earlier run finished.
- **Run:** the unchanged dataset flow.
- **Push:** only what this run produced: its source's file in each table, its clean audio and features, and its run record. The copy never replaces a file that is newer in S3, so runs of different sources can overlap without overwriting each other. If the flow fails, only its pipeline_runs row is pushed, so the failure is still recorded.

The Step Functions state machine ingests a dataset when its input names one, then always rebuilds the DuckDB warehouse from the whole lake, runs the data-quality report (v_clip_quality) and starts a Glue

crawl, so Athena sees any new tables. A failed step emails the team through SNS. Uploading team/recordings.csv to the raw bucket starts a team ingest by itself, which is the trigger from the "When the pipelines run" table in Part Three.

Access control and security

- **Private, encrypted storage:** every bucket blocks public access, encrypts objects at rest and refuses requests that do not use TLS. The data buckets also keep earlier versions of every object.
- **One role per job:** the pipeline task can read the raw bucket and read and write the lake bucket, and nothing else. The Glue crawler can only read the lake. Step Functions can run only our task definition, pass only its two roles and publish only to the alert topic.
- **No stored keys:** GitHub Actions and Modal sign in to AWS with short-lived OIDC tokens. Each GitHub role trusts only workflow jobs in our repository that run in its environment, and the Modal role trusts only our Modal workspace and training app.
- **Restricted CI permissions:** the GitHub role can push the pipeline image and read the account for terraform plan. Infrastructure changes are applied by a teammate after review, never by CI.
- **Network:** tasks can make outbound connections only; nothing can connect to them. S3 traffic stays inside AWS through a gateway endpoint.
- **Cost:** there is no NAT gateway and no server that runs between jobs, and a monthly budget emails the team at 80% of actual and 100% of forecast spend.

Alignment with Parts One to Three

- **Part One:** the Part Three pipeline preserves the core entities and field names from the aspirational schemas wherever the available data supports them.
- **Part Two:** each dataset we chose (VimSketch, the Vocal Imitation Set, the QVIM DEV set, ESC-50, ESC-50-Voice, CAPTDURE and our team recordings) gets its own folder in the raw bucket, in the form it was downloaded, when it is uploaded; prod currently holds ESC-50 and CAPTDURE. The raw buckets are kept private to avoid unnecessarily exposing project datasets.
- **Part Three:** the same bronze, silver and gold layers, folder layout, schemas and code run in the cloud. The upload trigger and the weekly schedule implement the "When the pipelines run" table, and the report after every run is the data-quality check from the warehouse design.

Testing

- **Terraform checks:** on every change under infra/, GitHub Actions runs terraform fmt, terraform validate on all four roots and our module tests. On main it also runs terraform plan against the dev environment with the read-only CI role.
- **Module tests:** seven terraform test runs use a mocked AWS provider, so they need no AWS account. They check the bucket names, the state machine's commands and steps, the upload trigger, each environment's task settings, that prod copies every bucket to the backup without copying deletes, that dev copies nothing, and that an unknown environment is rejected.
- **Pipeline tests:** twelve new tests cover copying the lake to and from S3, including a run against a local S3 server, and a full remote ingest where local folders stand in for the buckets. They check that a second run in a fresh container reprocesses nothing, and that two overlapping runs of different sources keep each other's tables.
- **Container image:** every pipeline change builds the image in CI and runs it once.
- **Live checks:** after deploying, we ran the dev pipeline end to end on CAPTDURE, checked that prod copies its buckets to the backup without copying deletes, and confirmed that GitHub Actions

and Modal can each sign in to their roles.

Part 5 Preparing for disaster recovery

Part Five deletes prod and restores it. Prod copies every new object in its three buckets to the backup stack in us-east-2 (Ohio), which has its own Terraform state, so destroying prod does not touch it. Deletes are not copied, so emptying the prod buckets leaves the backup intact. After terraform apply rebuilds prod and the pipeline-image workflow pushes the deployed commit's image again (the image registry is part of prod), we copy the buckets back from the backup and start the state machine, which rebuilds the warehouse, runs the data-quality report and registers the tables again. That last run also shows that the restored environment works.

Disaster-recovery run

We ran the Part Five demo on prod on October 24, following infra/disaster-recovery.md.
The recording is here:
https://drive.google.com/file/d/1XIDdQujOUCvr0Z8keOaurFHTuVWCdbva/view?usp=sharing

- **Before:** prod held 3,047 raw objects and 6,107 lake objects, the backup in us-east-2 held the same counts, and the last report showed 1,044 CAPTDURE clips and 2,000 ESC-50 clips (55 flagged as clipped).
- **Disaster:** terraform destroy removed all 78 prod resources, including its buckets and image registry. The backup buckets, which have their own state, kept every object.
- **Services, configuration and access:** terraform apply recreated the same 78 resources with the same names and ARNs, so the task definition, the bucket policies and the IAM roles came back unchanged, and neither the GitHub prod environment nor the Modal secret needed editing. The pipeline-image workflow then pushed the image for the current main commit. Only infrastructure and documentation had changed since the first deploy, so the pipeline code was the same.
- **Data:** copying the three buckets back from us-east-2 restored the same object counts, including the DuckDB warehouse file.
- **Verification:** a state-machine run on the restored lake rebuilt the warehouse, produced the same report as before the disaster and registered all 9 tables in Glue again. The run took under two minutes.
- **Lessons:** the Glue crawler keeps running after the state machine finishes, so the full table list appears a minute later; the image workflow's environment field defaults to dev, so the prod push has to select prod; and the SNS topic is recreated, so the alert subscription has to be confirmed again.

Works Cited

Cartwright, M., & Pardo, B. (2015). VocalSketch: Vocally imitating audio concepts. In Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (pp. 43–46). Association for Computing Machinery. https://doi.org/10.1145/2702123.2702387

Fonseca, E., Favory, X., Pons, J., Font, F., & Serra, X. (2020). FSD50K (Version 1.0) [Data set]. Zenodo. https://doi.org/10.5281/zenodo.4060432

Kim, B., Cartwright, M., Pishdadian, F., & Pardo, B. (2019). VimSketch dataset (Version 1.0) [Data set]. Zenodo. https://doi.org/10.5281/zenodo.2596911

Kim, B., Ghei, M., Pardo, B., & Duan, Z. (2018). Vocal Imitation Set: A dataset of vocally imitated sound events using the AudioSet ontology. In Proceedings of the Detection and Classification of Acoustic Scenes and Events 2018 Workshop (DCASE2018) (pp. 148–152). https://interactiveaudiolab.github.io/assets/papers/DCASE2018_Kim.pdf

Kim, B., & Pardo, B. (2018). Vocal Imitation Set v1.1.3: Thousands of vocal imitations of hundreds of sounds from the AudioSet ontology (Version 2) [Data set]. Zenodo. https://doi.org/10.5281/zenodo.1340763

Okamoto, Y., Imoto, K., Takamichi, S., Nagase, R., Fukumori, T., & Yamashita, Y. (2024). ESC-50-Voice: Dataset of vocal imitation for environmental sound in ESC-50 [Data set]. Zenodo. https://doi.org/10.5281/zenodo.11385662

Okamoto, Y., Shimonishi, K., Imoto, K., Dohi, K., Horiguchi, S., & Kawaguchi, Y. (2023). CAPTDURE: Captioned sound dataset of single sources [Data set]. Zenodo. https://doi.org/10.5281/zenodo.7965763

Paine, B. (n.d.). FreeSound.org LAION-640k dataset [Data set]. Hugging Face. Retrieved October 24, 2026, from https://huggingface.co/datasets/benjamin-paine/freesound-laion-640k

Piczak, K. J. (2015). ESC-50: Dataset for environmental sound classification [Data set]. GitHub. https://github.com/karolpiczak/esc-50

Pishdadian, F., Kim, B., Seetharaman, P., & Pardo, B. (2019). Classifying non-speech vocals: Deep vs signal processing representations. In Proceedings of the Detection and Classification of Acoustic Scenes and Events 2019 Workshop (DCASE2019) (pp. 194–198). https://interactiveaudiolab.github.io/assets/papers/pishdadian_kim_seetharaman_pardo_dcase2019.pdf

Südholt, D., Reiss, J., Zong, Y., Lu, M., Swanson, B., & Garcia Sihuay, N. (2025). MR20VI (Version 1) [Data set]. Zenodo. https://doi.org/10.5281/zenodo.17453307